outline and descibe what is needed in oder to finish semester

To finish Semester, you need to turn its broad product vision into a **verified** operating system: complete the connected workflows, prove permissions and data correctness, make the system reliable to operate, and obtain authorization before activating consequential institutional capabilities.

The next step is not another feature brainstorm. It is a completion program with one capability register, clear owners, evidence requirements, dependency order, and release gates.

Your project files already describe this distinction: the operations roadmap explicitly labels its scope as design-stage until code, tests, and release evidence exist; the algorithm catalog separates "catalogued" capabilities from "designed / logic drafted." Neither state establishes production readiness. [1] [2]

1. What "finished" should mean

Semester needs several finish lines—not one misleading "100% complete" label.

Finish line	What it means
Product-complete	The agreed workflows exist end to end, including failure and recovery paths
Technically verified	Tests, permissions, data reconciliation, accessibility, and security evidence support those workflows
Operationally ready	Monitoring, support, ownership, incident response, recovery, and rollback work
Institution-authorized	The institution has approved the relevant data access and official actions
Production-enabled	The approved capability is deployed and deliberately activated for the correct users
Replacement-ready	A domain can move from a legacy system to Semester without losing correctness, records, or operational continuity

Preserve the full Education OS ambition. Sequence implementation and activation by dependency and risk rather than removing entire domains from the vision.

Current-state limitation

I inspected relevant project files and checked the connected repository inventory. However, the connector did not expose the implementation-status file's text, and direct retrieval failed. I therefore cannot responsibly give a completion percentage or claim that a particular live feature is missing.

The requirements below are a completion framework grounded in your specifications—not a verified inventory of everything still unimplemented.

2. First: establish one authoritative completion register

Before adding more code, reconcile the existing product, repository, database, deployed environments, and documentation.

Your role-system specification already instructs implementers to inspect routes, schemas, migrations, policies, functions, tests, and deployment configuration rather than invent duplicate systems.[3]

Every capability needs a record

Field	Required information
Capability	A specific user outcome—not merely a screen name
Users and authority	Who may perform it and under whose authority
Existing implementation	Exact code, schema, route, and workflow references
Status	Designed, implemented, tested, deployed, authorized, enabled
Dependencies	Identity, data, policies, integrations, and other workflows
Evidence	Test results, permission proof, reconciliation, accessibility review
Owner	Accountable person and backup
Blockers	Missing decisions, code, access, validation, or approval
Activation gate	What must be true before users can access it
Recovery	Retry, correction, rollback, support, and data restoration
Acceptance criteria	Observable conditions establishing completion

Completion-register rules

- A screen is not a completed workflow.
- A migration file is not proof it was applied.
- A passing build is not proof of correct authorization.
- A deployed route is not proof of institution approval.
- A local prototype is not a production integration.
- A generated document is not implemented product behavior.
- A feature flag is not a substitute for backend authorization.

The recent interview workbooks, grading templates, and reconciliation scripts should enter this register as local artifacts with their actual verification scope—not as completed Semester career infrastructure.

3. Complete the shared platform foundation

Every domain depends on this layer. Without it, Semester becomes a collection of disconnected applications.

Your role-lifecycle specification defines one identity, policy, data, workflow, audit, and intelligence layer, with purpose-built workspaces for each role.[4]

Required foundation

Area	Completion requirement
Identity	Signup, verification, recovery, institution linking, SSO, account changes, deprovisioning
Tenant model	Institution hierarchy, memberships, roles, scoped capabilities, cross-tenant denial
Authorization	Server, database, storage, export, and background-job checks
Education graph	Canonical relationships among people, courses, terms, requirements, artifacts, services, and opportunities
Workflow engine	States, transitions, approvals, receipts, retries, escalation, and corrections
Action Center	One coherent queue with ownership, deadlines, status, and completion evidence
Search	Permission-aware retrieval with source authority and freshness
Files and workspace	Ownership, versioning, sharing, retention, export, and deletion
Notifications	Preferences, deduplication, quiet controls, delivery status, and escalation
Design system	Shared navigation, components, language, status patterns, and accessibility
Commercial controls	Entitlements and billing boundaries without exposing student records to company roles

Acceptance test

A user should move between planning, coursework, advising, career, and services without losing identity, context, permissions, or task state.

A role change or consent revocation should remove access across all affected surfaces—not just hide a button.

4. Finish the student daily loop

This should be the first complete product experience.

End-to-end student journey

```
Create or link account
→ establish term and goals
→ bring in permitted course information
→ understand today's priorities
→ plan manageable work
→ study or complete a task
```

```
→ receive feedback or support
→ update the plan
→ recover after interruption
→ archive or transition
```

Workflows that need completion evidence

- Role-aware onboarding and a first meaningful action.
- Course and deadline setup.
- Calendar and workload planning.
- Action Center prioritization.
- Study workspace using approved sources.
- Assignment progress and submission handoff.
- Support requests and human referrals.
- Registration preparation.
- Career evidence capture.
- Term rollover, leave, graduation, and export.

The project's student lifecycle connects Today, Action Center, Calendar, Courses, Study, and Support to cross-role workflows and lifecycle transitions. [4]

Hardening requirements

- Interrupted onboarding resumes correctly.
- Failed imports do not create misleading deadlines.
- Offline edits have clear saved/pending states.
- Duplicate source events do not multiply tasks.
- Time zones and changed deadlines reconcile correctly.
- Returning after a missed week produces a manageable recovery plan.
- Students can override personal planning suggestions.
- Private work stays separate from official records.

"Finished" means these behaviors survive disruption—not merely that the happy path looks polished.

5. Complete every product domain end to end

A. Learning, faculty, and teaching

Needed:

- Course setup and policy.
- Materials and assignments.

- Submission handling.
- Rubrics and feedback.
- Teaching-team delegation.
- Office hours and communication.
- AI-assisted assessment with human authority.
- Moderation, release, regrade, and correction.
- Grade handoff and term archive.

The project defines assessment as a workflow from policy and accessibility checks through submissions, feedback, moderation, release, audit, and learning evidence. [4]

Completion proof: a real assignment can move from creation to student submission, reviewed feedback, authorized release, appeal, and archive without bypassing permissions.

B. Academic planning and registration

Needed:

- Program requirements and provenance.
- Prerequisite and sequencing logic.
- Degree scenarios with explicit assumptions.
- Sections, schedules, alternatives, and conflicts.
- Holds and registration windows.
- Official enrollment or controlled handoff.
- Waitlists, overrides, receipts, and reconciliation.
- Records, graduation, and credential workflows.

Your registration-readiness specification requires each item to show its source, timestamp, and action path, with “information unavailable” and “needs advisor review” as valid outcomes. [5]

Completion proof: stale or missing data cannot produce a false “ready” state, and official enrollment is never confused with a saved personal plan.

C. Advising and student support

Needed:

- Scoped caseloads.
- Appointments.
- Student-controlled context sharing.
- Plans and referrals.
- Outreach and follow-up.
- Responsible-office routing.

- Accepted handoffs.
- Resolution and escalation.
- Aggregate outcomes with appropriate privacy.

Completion proof: a request reaches an accountable owner, the receiving service acknowledges it, and the student can understand its status.

D. Career, employer, and alumni

Needed:

- Exploration and recommendations.
- Skills and evidence.
- Résumé and portfolio.
- Opportunities and applications.
- Interview preparation and practice.
- Human coaching.
- Employer-approved interactions.
- Mentoring.
- Offer and transition support.
- Voluntary outcomes and lifelong continuity.

Your career handoff requires employer access to approved evidence only, tested revocation, and institution-specific feature switches.[6]

Completion proof: the student can prepare, practice, share deliberately, receive coaching, and revoke access without private practice becoming an employer assessment.

E. Campus, community, and family

Needed:

- Clubs, events, spaces, and service discovery.
- Membership and bookings.
- Community reporting and moderation.
- Appeals and safety handoffs.
- Family invitations and scoped grants.
- Grant expiry, access history, and revocation.
- Housing, dining, library, and other service operations.

Completion proof: permissions and service ownership remain clear across the entire interaction, including reporting, cancellation, and consent changes.

F. Student accounts and institutional finance

Needed:

- Charges, credits, statements, and reconciliation.
- Payment-plan workflows.
- Authorized payment processing.
- Aid and accounts handoffs.
- Refund and correction controls.
- Separation of student finance from Semester commercial billing.
- Broader financial operations where institution-authorized.

Completion proof: amounts reconcile to their authoritative source, consequential actions have proper approval, and failures do not leave inconsistent financial records.

These domains should remain in the master scope while their production activation follows legal, financial, security, and institutional gates.

6. Finish Semester Intelligence as governed infrastructure

Do not treat the AI layer as “done” because chat works.

The algorithm catalog includes next-action ranking, recommendations, search, support routing, provider routing, confidence/escalation, moderation assistance, and reconciliation. Many entries are explicitly catalogued or logic-drafted rather than production-verified. [2]

Required AI capabilities

- Authenticated, tenant-aware requests.
- Purpose and policy checks.
- Student-selected context.
- Approved provider and model routing.
- Permission-filtered retrieval.
- Source provenance and freshness.
- Uncertainty and unavailable-data behavior.
- Bounded tools and approval requirements.
- Human escalation.
- Evaluation datasets and versioned rubrics.
- Cost, latency, failure, and incident monitoring.
- Disable controls and provider fallback.

Required behavior by role

Role	Assistance	Authority boundary
Student	Planning, study, exploration, practice	Student controls choices and sharing
Faculty	Drafting, rubric feedback, grading assistance	Educator owns official assessment
Advisor	Summaries, routing, preparation	Human owns consequential advising decisions
Registrar	Validation assistance and workflow preparation	Authorized process owns official records
Career staff	Practice feedback and coaching support	No hidden employability scoring
Operations	Incident summaries and approved workflow assistance	No unrestricted cross-tenant access

Policy blocker to resolve

Some execution handoffs say AI never receives student records, while the broader role specification permits policy-governed contextual access. Those instructions need one explicit policy interpretation before implementation. Do not silently choose the more permissive version. [7] [3]

7. Complete integrations and replacement machinery

Semester needs an integration system—not a separate script for every vendor.

Each integration needs

- Approved access and scopes.
- Source and destination ownership.
- Versioned mappings.
- Stable identifiers.
- Validation.
- Idempotency.
- Retry limits.
- Reconciliation.
- Error queues and alert routing.
- Freshness indicators.
- Revocation and deactivation.
- Export and portability.
- Operational documentation.

The operations roadmap already includes an integration registry, sync status, mapping versions, and reconciliation as foundational work. [1]

Domain replacement sequence

Your workflow catalog defines:

Observe → shadow → assist → dual run → gate → authoritative → archive → retire.[8]

Before any cutover, require:

- Record-count and field-level reconciliation.
- Workflow comparison.
- Permission equivalence.
- Historical-data validation.
- Exception handling.
- Recovery rehearsal.
- Institutional acceptance.
- An identified authoritative source during each phase.

An API connection is not proof that Semester can replace the connected system.

8. Finish trust, reliability, and operations

These are product requirements, not launch paperwork.

Trust and security

- Tenant and capability enforcement.
- Least privilege and access reviews.
- Restricted support access.
- Secure credentials and secret rotation.
- Storage and export authorization.
- Audit records.
- Data classification and minimization.
- Retention, legal holds, deletion, and portability.
- Appropriate vendor and contractual reviews.
- Accessibility verification.
- Incident handling and vulnerability remediation.

Reliability

- Defined service objectives.
- Monitoring and actionable alerts.
- Durable queues and background-job visibility.

- Backups and tested restores.
- Recovery and rollback procedures.
- Deployment and migration controls.
- Dependency-failure handling.
- Support escalation and ownership.
- Status communication.

Your operations release gates require cross-tenant denial tests, controlled high-impact actions, source labels, accessibility checks, implemented screen states, monitoring, support ownership, and tested rollback.[9]

Recovery targets must be selected and validated for the actual deployment. Do not claim an uptime or recovery guarantee merely because it appears in a specification.

Company and institutional operations

Finish:

- Tenant setup and configuration.
- Approvals and operator work queues.
- Support and escalation.
- Pilot and implementation tracking.
- Release and incident management.
- Billing and entitlements.
- Institution training.
- Customer outcomes and renewal.
- Offboarding and verified export.

The operations roadmap explicitly warns about parallel truths, broad admin access, and a solo-founder approval bottleneck. Backup approvers and ownership are therefore part of completion.[1]

9. The universal acceptance gate

Apply this to every enabled capability.

Gate	Evidence required
User outcome	End-to-end workflow succeeds
Authorization	Tenant, role, relationship, and resource restrictions are tested
Correctness	Data and state transitions reconcile
Failure handling	Errors, interruption, retries, and recovery behave clearly
Accessibility	Relevant keyboard, screen-reader, visual, and alternative-flow checks pass

Gate	Evidence required
Privacy	Data use, sharing, retention, and revocation are implemented
AI quality	Applicable evaluations and human boundaries pass
Operations	Monitoring, support owner, runbook, and escalation exist
Reversibility	Rollback or approved remediation is tested
Documentation	Implementation and user-facing claims match reality
Activation	Authorized owner approves the actual release scope

The launch handoff requires evidence for every go/no-go gate and a rollback rehearsal in staging. Its reference to a Vanderbilt pilot is a planning artifact—not proof of an institutional agreement or live pilot.[7]

10. Execution order to finish without fragmenting the product

Stage 1 — Audit and reconcile

Owner: engineering lead with product and institutional implementation.

Deliver:

- Capability register.
- Existing implementation map.
- Evidence gaps.
- Canonical-model decisions.
- Duplicate-system decisions.
- Risk and dependency register.
- Named owners and backups.

Exit gate: the team can distinguish what is designed, implemented, tested, deployed, authorized, and enabled.

Stage 2 — Harden the common foundation and daily loop

Owner: platform and student-experience leads.

Deliver:

- Identity and authorization.
- Shared workflow infrastructure.
- Action Center, calendar, search, and workspace.
- Reliable student onboarding and recovery.
- Policy-controlled AI.

- Monitoring and support.

Exit gate: representative users complete the daily loop under normal and failure conditions.

Stage 3 — Complete cross-role workflows

Owner: domain leads.

Deliver complete student–faculty, student–advisor, registration, career, campus, accounts, and administration interactions.

Exit gate: both sides of each workflow work, including approval, correction, escalation, and lifecycle changes.

Stage 4 — Validate institutional operation

Owner: implementation, security, privacy, accessibility, and operations.

Deliver:

- SSO and provisioning.
- Approved integration access.
- Reconciled data.
- Training.
- Support coverage.
- Recovery rehearsal.
- Institutional acceptance.

Exit gate: an authorized launch decision with attached evidence.

Stage 5 — Enable and replace by domain

Owner: institutional domain authority and migration lead.

Deliver monitored rollout, dual operation, reconciliation, controlled cutover, and retirement of the prior system where justified.

Exit gate: Semester is the accepted authoritative system for that specific domain—not simply another interface over it.

11. What to do next

The highest-value next artifact is a single evidence-linked completion backlog—not another disconnected prototype.

Start with these actions:

1. Reconcile the repository’s status, completeness, evidence, and go/no-go documents.
2. Map each capability to actual code, database objects, routes, and tests.

3. Compare deployed schema and configuration with repository intent.
4. Identify duplicates and choose the canonical implementation.
5. Assign owners and backups.
6. Select the student daily loop as the first full verification path.
7. Close authorization and data-correctness gaps before enabling more writes.
8. Resolve the AI student-record policy conflict.
9. Run applicable tests and record their actual results.
10. Rehearse failure, restore, rollback, and offboarding.
11. Establish institutional activation decisions by domain.
12. Publish a status dashboard based on evidence rather than feature counts.

Do not estimate time to finish from the number of screens. Estimate it after this audit, using workflow complexity, dependencies, verification effort, staffing, and external approvals.

Semester is finished for a release when its promised workflows operate as one coherent system, its authority boundaries are enforced, and its evidence supports what users and institutions are told it can do.

⁂

1. The Main Semester design system (2) copy/docs/operations/OPERATIONS_ROADMAP.md
2. The Main Semester design system (2) copy/docs/system/algorithms/SEMESTER_ALGORITHM_CATALOG.md
3. The Main Semester design system (2) copy/handoff/specifications/role-system-execution-prompt.pdf
4. The Main Semester design system (2) copy/docs/system/ROLE_LIFECYCLES.md
5. The Main Semester design system (2) copy/docs/system/algorithms/SEMESTER_REGISTRATION_READINESS.md
6. The Main Semester design system (2) copy/handoff/execute/07-community-career-family.md
7. The Main Semester design system (2) copy/handoff/execute/10-launch.md
8. The Main Semester design system (2) copy/docs/operations/OPERATIONS_WORKFLOW_CATALOG.md
9. The Main Semester design system (2) copy/docs/operations/OPERATIONS_RELEASE_GATES.md